

Reviewer Pack — Plethysm Coefficient Gap in Permanent vs Determinant Decompo...

Entry 40591186a3ea · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-12 01:34:49 UTC

Plethysm Coefficient Gap in Permanent vs Determinant Decompositions

Entry ID: 40591186a3ea **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-12 01:34:49 UTC

1. Conjecture

Field A (mathematical branch): Schur-Weyl Duality **Field B** (complexity object): Boolean Circuit Size for Permanent Polynomial

Statement:

For all $n \geq 2$, the plethysm coefficient $\lambda(\text{perm}_n)$ of the symmetric power $S^k(\text{perm}_n)$ exceeds $\lambda(\det_m) + \varepsilon \cdot n^2$ for $m < n^{\{1.5\}}$, where $\varepsilon > 0$ is universal. This gap persists under linear substitutions preserving the polynomial’s symmetry type.

Rationale (proposer’s reasoning):

Plethysm coefficients encode symmetry constraints of polynomial representations. By linking them to circuit complexity, we may expose algebraic barriers to expressing permanent as a determinant-like circuit, leveraging representation-theoretic invariants resistant to natural proofs.

Taxonomy category: GCT_DET_PERM (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c495be0631aa1f56

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.2s

5.1 Generated Python source

```
import random
import math

def factorial(n):
    if n == 0:
        return 1
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result

def hook_length_formula(young_tableau):
    m, n = len(young_tableau), len(young_tableau[0])
    total = 0
    for i in range(m):
        for j in range(n):
            hook_length = (i + 1) + (n - j) - young_tableau[i][j]
            total += factorial(hook_length) // (factorial(i + 1) * factorial(n - j))
    return total

def generate_matrix(n):
    return [[random.randint(0, 10) for _ in range(n)] for _ in range(n)]

def plethysm_coefficient(matrix, k):
    n = len(matrix)
    young_tableau = []
    for i in range(n):
        row = [k - sum(matrix[i][j] for j in range(i + 1)) for j in range(i + 1)]
        young_tableau.append(row)
    return hook_length_formula(young_tableau)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    m = math.floor(n ** 1.5)

    perm_matrix = generate_matrix(n)
    det_matrix = generate_matrix(m)
```

```

lambda_perm_n = plethysm_coefficient(perm_matrix, n)
lambda_det_m = plethysm_coefficient(det_matrix, m)

epsilon = 0.1
if lambda_perm_n > lambda_det_m + epsilon * n ** 2:
    conjecture_holds = True
    counterexample = ""
else:
    conjecture_holds = False
    counterexample = "plethysm_coefficient_gap"

return {
    "metric_name": "plethysm_coefficient_gap",
    "metric_value": lambda_perm_n - (lambda_det_m + epsilon * n ** 2),
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]]
    if not seeds:
        from sympy import primerange
        seeds = list(primerange(2, 30))[:30]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(res["metric_value"] for res in results) / len(results)
    std_value = math.sqrt(sum((res["metric_value"] - mean_value) ** 2 for res in results) / len(results))
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value:.4f} std={std_value:.4f} support_fraction={support_fraction:.4f}")
    elif any(not res["conjecture_holds"] for res in results):
        first_failing_seed = next(seed for seed, res in zip(seeds, results) if not res["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{res['counterexample']}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d9d81969.py", line 81, in <module>
    result = run_trial(seed)
            ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d9d81969.py", line 58, in run_trial

```

```

if lambda_perm_n > lambda_det_m + epsilon * n ** 2:
    ~~~~~^~~~~~

```

OverflowError: int too large to convert to float

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with overflow error before producing data; cannot verify conjecture's validity
| next: Implement arbitrary-precision arithmetic or numerical stability checks for large n

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	112240
2	propose	ollama_remote	qwen3:8b	0	0	28521
3	preregistration	ollama_remote	qwen3:8b	0	0	24186
4	novelty	ollama_remote	qwen3:8b	0	0	20806
5	novelty	ollama_remote	qwen3:8b	0	0	14723
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15203
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8911
8	judge	ollama_remote	qwen3:8b	0	0	21521

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 246110 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in *research/audit/40591186a3ea.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/40591186a3ea.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/40591186a3ea.tar.gz* (if generated)